

Triangle Problem - Algorithm Comparison

Problem

Given an array A of N integers, return 1 if there exists a triangular triplet (P, Q, R) where:

- $A[P] + A[Q] > A[R]$
- $A[Q] + A[R] > A[P]$
- $A[R] + A[P] > A[Q]$

Insight

After **sorting**, only one condition needs to be checked: $A[i] + A[i+1] > A[i+2]$, and only for **consecutive triplets** — because consecutive elements give the best chance of satisfying the inequality.

Comparison

	Iterative	Recursive
Sorting Algorithm	Insertion Sort	Merge Sort
Scan	for loop	Recursive calls
Sort Complexity	$O(n^2)$	$O(n \log n)$
Scan Complexity	$O(n)$	$O(n)$
Overall Complexity	$O(n^2)$	$O(n \log n)$
Recurrence (scan)	None	$T(n) = T(n-1) + O(1) \rightarrow \Theta(n)$

Iterative Solution

```
int triangle(vector<int> a) {
    int n = a.size();
    insertionSort(a);
    for (int i = 0; i < n - 2; i++) {
```

```
        if (a[i] + a[i + 1] > a[i + 2])
            return 1;
    }
    return 0;
}
```

Recursive Solution

```
int triangle(const vector<int> &A, int i)
{
    if (i >= A.size() - 2)
        return 0;

    if (A[i] + A[i + 1] > A[i + 2])
        return 1;

    return triangle(A, i + 1);
}
```

Conclusion

Both solutions use the same logic as in sorting and then scanning the array. The difference is between the sorting algorithms used for each solution. The recursive solution uses Merge sort which is $O(n \log n)$ making the overall complexity $O(n \log n)$. The iterative solution uses Insertion Sort which is $O(n^2)$ making the overall complexity $O(n^2)$. So the winner is recursive solution for larger inputs.